

LeetCode 1478: Allocate Mailboxes

Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 28, 2026

Asked in: Adobe, Apple

1. Problem Understanding

$houses[i]$ → Location of the i^{th} house along a street. $\in [1, 10^4]$

Note: No 2 houses are at the same place.

k = number of mailboxes to allocate.

Objective: Need to return the sum of the minimum hamming distance of each house from a mailbox.

2. Example Breakdown

Array: $[1, 4, 8, 10, 20]$ | $k = 3$

Placing mailboxes at optimal positions (e.g., 3, 9, 20):

Distances for each house: $2 + 1 + 1 + 1 + 0 = 5$

3. Core Idea & Logic

Basic idea ye he ki:

Pehle $[l....r]$ ke range of houses ko bas 1 mailbox do. Ye isliye required he kyunki $[0....j-1][j....i-1]$ me jo last part he, usko 1 mailbox milega. Baaki ke pehle waale serve ho jaayenge baaki mail box se.

And since 1 mailbox dena he $[l....r]$ ko, simply **place in median** house.

Will build a $cost[l][r]$ to precompute hamming distance if placed at median.

Let the first $(k-1)$ mails be placed optimally, then you only need to place the last mail for the last block.

4. C++ Implementation

```

class Solution {
public:
    int minDistance(vector<int>& houses, int k) {
        int n=houses.size();
        sort(houses.begin(),houses.end());

        //basic aadya ye he ki
        //pehle [1....r] ke range of houses ko bas 1 mailbox do
        //ye isliye required he kyunki

        //[0.....j-1][j....i-1] me jo last part he , usko 1 mailbox milega
        //baaki ke pehle waale serve ho jaayenge baaki mail box se

        //and since 1 mailbox dena he [1....r] ko simply place in median

        vector<vector<int>> cost(n,vector<int>(n,0));
        for(int l=0;l<n;l++){
            for(int r=l;r<n;r++){
                int med=(r+l)/2;
                int mail=houses[med];
                int currcost=0;

                for(int i=l;i<=r;i++){
                    currcost+=abs(houses[i]-mail);
                }

                cost[l][r]=currcost;
            }
        }

        //ab partition dp start

        vector<vector<int>> dp(n+1,vector<int>(k+1,INT_MAX));
        dp[0][0]=0;
        for(int i=1;i<=n;i++){
            for(int parts=1;parts<=k;parts++){
                for(int j=parts-1;j<i;j++){

                    if(dp[j][parts-1]==INT_MAX) continue;
                    int curr=dp[j][parts-1]+cost[j][i-1];
                    dp[i][parts]=min(dp[i][parts],curr);
                }
            }
        }
    }
};

```

```
        }  
    }  
  
    return dp[n][k];  
  
}  
};
```